

Bentix Architecture

Visão Geral

A Bentix é uma plataforma de gestão operacional para equipas e obras. A aplicação concentra num único produto:

- autenticação e sessão de utilizador
- gestão de pessoas e identidades de acesso
- clientes e obras
- planeamento diário e afetações
- registo, submissão e aprovação de horas
- materiais, notificações e ferramentas técnicas de suporte

Stack tecnológica

- `Next.js 16` com `App Router`
- `React 19`
- `Node.js 22` em Docker
- `Prisma 7` com `@prisma/adapter-mariadb`
- `MariaDB 11.4`
- `Nginx` como reverse proxy
- `Cloudflare` para DNS e edge proxy
- `Swagger UI` / OpenAPI para documentação da API
- `node:test` para testes críticos automatizados

Princípios da arquitetura

- monorepo único para frontend, API HTTP e acesso a dados
- rotas `app/api/*` finas, sem lógica de negócio relevante
- separação clara entre `controllers`, `services` e `lib`
- configuração pública centralizada por ambiente em `config/`
- autenticação por cookie HTTP-only e sessão assinada
- MySQL/MariaDB como fonte operacional principal
- preparação para futura separação real entre frontend e backend, sem partir contratos atuais

Arquitetura Geral

Diagrama lógico principal:

```
graph TD; Browser --> Web["Web (Next.js)"]; Web --> REST_API["REST API"]; REST_API --> MariaDB;
```

Papel de cada camada

Browser

O browser executa a interface React entregue pelo Next.js, mantém o cookie de sessão e faz chamadas `fetch` para a API com `credentials: include`.

Web (Next.js)

O Next.js cumpre dois papéis:

- serve as páginas e componentes da aplicação
- expõe a API em `app/api/*`
- executa `proxy.js` para guards de acesso, redireção HTTPS em produção e preflight/CORS das rotas API

Na prática, a app continua monolítica. Mesmo quando frontend e API são publicados em subdomínios diferentes, o código continua no mesmo repositório e a mesma imagem Docker é reutilizada.

REST API

A API usa a seguinte cadeia interna:

```
graph TD; route["app/api/*/route.js"] --> controllers["server/controllers/*"]; controllers --> services["server/services/*"]; services --> db["lib/* + lib/db/*"];
```

As `route.js` recebem o pedido HTTP, os `controllers` tratam do boundary HTTP, os `services` concentram regras aplicacionais e `lib/db/*` fala diretamente com Prisma.

MariaDB

MariaDB é a base de dados persistente alvo da aplicação. O acesso é feito por Prisma e centralizado em `lib/prisma.js` e `lib/prisma-adapter.js`.

Diagrama de deployment atual



Ambientes

A arquitetura suportada é apenas:

- LOCAL
- DEV
- PROD

Resumo por ambiente

Ambiente	Frontend	API	Base de dados	Execução
LOCAL	<code>http://localhost:3000</code>	mesma origem em <code>/api/*</code>	MySQL/MariaDB local, tipicamente <code>bentix_local</code>	<code>npm run dev</code>
DEV	<code>https://dev.bentixapp.com</code>	<code>https://api-dev.bentixapp.com</code>	MariaDB Docker <code>bentix_dev</code>	<code>docker compose</code>
PROD	<code>https://bentixapp.com</code>	<code>https://api.bentixapp.com</code>	MariaDB Docker <code>bentix_prod</code>	<code>docker compose</code>

LOCAL

- o frontend e a API correm no mesmo processo Next.js
- o browser usa URLs relativas `/api/*`
- não existe separação por subdomínio
- o cookie pode ser host-only, sem `Domain`
- a base de dados é configurada por `DATABASE_URL`

DEV

- o frontend é publicado em `dev.bentixapp.com`
- a API é publicada em `api-dev.bentixapp.com`
- o `docker-compose` cria três serviços físicos: `web`, `api` e `db`
- `web` e `api` usam a mesma app Next.js, mas com propósito de publicação diferente
- para partilha de sessão entre subdomínios, o cookie deve usar `SESSION_COOKIE_DOMAIN=.bentixapp.com`

PROD

- o frontend é publicado em `bentixapp.com`
- a API é publicada em `api.bentixapp.com`
- hoje o `docker-compose` de produção tem dois serviços físicos: `app` e `db`
- logicamente existem os papéis `web` e `api`, mas ambos são expostos pelo mesmo serviço `app` atrás do Nginx
- isto mantém a arquitetura atual simples sem alterar a lógica da aplicação

Configuração por ambiente

A configuração pública da URL da API está em:

- `config/app.local.js`
- `config/app.dev.js`

- `config/app.prod.js`
- `config/app.public.js`

Regras:

- `NEXT_PUBLIC_APP_ENV` aceita apenas `local`, `dev` ou `prod`
- `NEXT_PUBLIC_API_BASE_URL`, quando definido, faz override da URL do perfil
- sem override, a app usa a URL definida em `config/app.<ambiente>.js`

Estrutura da aplicação

```
frontend/  
app/  
server/  
lib/  
config/  
infra/  
tests/
```

`frontend/`

Camada cliente para consumo da API a partir de componentes React.

- `frontend/api/api-client.js` resolve a base URL e executa `fetch`
- `frontend/controllers/*` encapsulam chamadas à API e evitam espalhar detalhes de transporte pela UI

`app/`

Camada Next.js App Router.

- páginas e layouts
- componentes React server/client
- rotas HTTP em `app/api/*`
- Swagger UI em `/developer/api-docs`

`server/`

Camada HTTP e aplicação.

- `server/controllers/*`: parsing de requests, sessão, autorização, mapeamento de erros
- `server/services/*`: lógica de aplicação e orquestração
- `server/responses/*`: helpers de resposta HTTP
- `server/errors/*`: erros tipados
- `server/docs/*`: OpenAPI servido pela própria aplicação

lib/

Camada transversal de domínio e infraestrutura interna.

- autenticação, cookies e sessão
- permissões, roles e perfis de acesso
- acesso a dados por entidade
- helpers de negócio reutilizáveis
- compatibilidade legacy e alguns fallbacks técnicos

Dentro de `lib/` :

- `lib/db/*` contém a camada de acesso a dados por entidade
- `lib/prisma.js` e `lib/prisma-adapter.js` inicializam Prisma
- `lib/data-source.js` escolhe entre `mysql` e fallback `json`

config/

Configuração pública da aplicação por ambiente.

- seleciona a base URL da API consumida pelo frontend
- isola a configuração de ambiente do código de negócio

infra/

Infraestrutura versionada.

- `infra/environments/*` : `docker-compose` e `.env.example`
- `infra/docker/app/Dockerfile` : build e runtime da imagem da app
- `infra/nginx/*` : reverse proxy por ambiente
- `infra/scripts/*` : utilitários operacionais

tests/

Testes críticos automatizados.

- fluxo de autenticação
- CORS
- configuração pública por ambiente
- opções do cookie de sessão
- fluxos críticos de planeamento e afetações

Fluxo de autenticação

Fluxo completo:

```
Browser
|
▼
GET /login
|
▼
GET /api/auth/payload-key
|
▼
Browser cifra credenciais
|
▼
POST /api/auth/login
|
▼
Cookie bentix_session
|
▼
GET /api/auth/session e restantes rotas
|
▼
GET /api/auth/logout
|
▼
Cookie expirado
```

Passo a passo

1. O utilizador abre `/login`.
2. O frontend pede a chave pública em `/api/auth/payload-key`.
3. O browser cifra o payload de login com AES-GCM e cifra a chave efémera com RSA-OAEP.
4. O frontend envia `POST /api/auth/login` com `protectedPayload`.
5. `server/controllers/auth-login-controller.js` lê o corpo protegido.
6. `lib/login-transport.js` decifra o payload.
7. `server/services/auth-login-service.js` valida username/password, verifica bloqueios, atualiza auditoria e cria o token de sessão.
8. `lib/auth.js` assina a sessão com `AUTH_SECRET` e devolve um token HMAC.
9. A resposta define o cookie `bentix_session` com `HttpOnly`, `Path=/` e política de `SameSite / Domain` dependente do ambiente.
10. Nas chamadas seguintes, o browser envia o cookie automaticamente porque `frontend/api/api-client.js` usa `credentials: include`.
11. `proxy.js` e `lib/server-session.js` validam a sessão antes de permitir acesso a páginas e endpoints protegidos.

12. Em `/api/auth/logout`, a aplicação expira o mesmo cookie com as mesmas opções base.

Sessão e reidratação

A aplicação não confia apenas no cookie bruto. Depois de validar a assinatura, `lib/server-session.js` volta a carregar o utilizador e o estado de acesso a partir da fonte de dados ativa.

Isto permite:

- invalidar contas desativadas
- recalcular permissões e perfis de acesso
- atualizar `workIds` e identidade operacional

Cookie de sessão

Comportamento atual:

- `HttpOnly=true`
- `Path=/`
- `SameSite=Lax` em host local sem `SESSION_COOKIE_DOMAIN`
- `SameSite=None` quando existe `SESSION_COOKIE_DOMAIN`
- `Secure=true` em produção e sempre que existe `SESSION_COOKIE_DOMAIN`

Exemplo esperado para DEV/PROD com frontend e API em subdomínios distintos:

```
Domain=.bentixapp.com; SameSite=None; Secure; HttpOnly; Path=/
```

Fluxo de uma chamada REST API

Diagrama pedido:

```
Browser
↓
Web
↓
api-client
↓
REST API
↓
Service
↓
Prisma
↓
MariaDB
```


Explicação passo a passo

1. Browser

O utilizador interage com a página ou componente.

2. Web

O frontend Next.js renderiza a UI e aciona uma ação do utilizador. Em páginas client-side, isto acontece dentro de componentes React ou de `frontend/controllers/*`.

3. `api-client`

`frontend/api/api-client.js`:

- valida o path pedido
- resolve a base URL via `config/app.public.js`
- usa `NEXT_PUBLIC_APP_ENV` e `NEXT_PUBLIC_API_BASE_URL`
- executa `fetch(..., { credentials: 'include' })`

Em `LOCAL`, a URL final é relativa, por exemplo `/api/people`.

Em `DEV`, a URL final é tipicamente `https://api-dev.bentixapp.com/api/people`.

Em `PROD`, a URL final é tipicamente `https://api.bentixapp.com/api/people`.

4. REST API

O pedido entra numa `route.js` em `app/api/*`. A rota:

- chama um controller
- converte o resultado em `NextResponse`
- converte erros tipados em `status codes`

Antes de chegar ao controller, `proxy.js` pode:

- responder ao preflight `OPTIONS`
- aplicar headers CORS para chamadas cross-origin
- recusar pedidos sem sessão ou sem permissão

5. Service

O controller delega a lógica aplicacional para `server/services/*`.

Exemplo típico:

- valida sessão
- verifica permissões
- valida input
- orquestra operações

6. Prisma

Os `services` usam `lib/*` e `lib/db/*`, que por sua vez usam Prisma para falar com MariaDB.

7. MariaDB

MariaDB persiste o estado da aplicação: pessoas, utilizadores, obras, afetações, eventos, permissões e restantes entidades.

Exemplo real: GET /api/people

```
app/api/people/route.js
|
▼
server/controllers/people-controller.js
|
▼
server/services/people-service.js
|
▼
lib/people.js
|
▼
lib/db/people-db.js
|
▼
Prisma
|
▼
MariaDB
```

Configuração

Variáveis principais

Variável	Papel	Onde é usada	Leitura principal
<code>NEXT_PUBLIC_APP_ENV</code>	escolhe o perfil público <code>local/dev/prod</code>	<code>config/app.public.js</code> , frontend bundle	build time
<code>NEXT_PUBLIC_API_BASE_URL</code>	override explícito da base URL da API	<code>config/app.public.js</code> , frontend bundle	build time
<code>SESSION_COOKIE_DOMAIN</code>	define partilha do cookie entre subdomínios	<code>lib/auth.js</code>	runtime
<code>DATABASE_URL</code>	ligação Prisma/MariaDB	<code>lib/prisma.js</code> , <code>lib/prisma-adapter.js</code>	runtime
<code>AUTH_SECRET</code>	assinatura HMAC da sessão	<code>lib/auth.js</code>	runtime

`NEXT_PUBLIC_APP_ENV`

- valores suportados: `local` , `dev` , `prod`
- seleciona `config/app.local.js` , `config/app.dev.js` ou `config/app.prod.js`
- é embebida na app pelo processo de build

`NEXT_PUBLIC_API_BASE_URL`

- é opcional
- quando definido, tem prioridade sobre a URL do perfil
- é útil para forçar uma API alternativa sem mudar código

`SESSION_COOKIE_DOMAIN`

- em branco no `LOCAL` , para cookie host-only
- `.bentixapp.com` em `DEV` e tipicamente também em `PROD` , para partilhar sessão entre `web` e `api`

`DATABASE_URL`

- define a base de dados real usada pela aplicação em runtime
- também existe uma `DATABASE_URL` dummy no Docker build para permitir `prisma generate` e `next build`
- a dummy não deve ser confundida com a ligação real de produção

AUTH_SECRET

- segredo usado para assinar e validar o conteúdo da sessão
- obrigatório em produção
- sem este valor, a aplicação não consegue validar cookies de sessão de forma segura

Build time vs runtime

Build time

- NEXT_PUBLIC_APP_ENV
- NEXT_PUBLIC_API_BASE_URL

Runtime

- DATABASE_URL
- AUTH_SECRET
- SESSION_COOKIE_DOMAIN
- LOGIN_PUBLIC_KEY_PEM
- LOGIN_PRIVATE_KEY_PEM

Nota importante:

- no Docker atual, `NEXT_PUBLIC_*` entra como `ARG` e `ENV` antes de `next build`
- por isso, para efeitos do browser, a configuração pública deve ser tratada como configuração de build

Docker

Imagem da aplicação

O Dockerfile em `infra/docker/app/Dockerfile`:

- usa `node:22-bookworm-slim`
- instala dependências
- executa `npm run db:generate`
- executa `npm run build`
- copia `.next`, código e artefactos necessários para a imagem final

DEV

Topologia física atual:

```
web  -> frontend publicado em dev.bentixapp.com
api  -> mesma app Next.js publicada em api-dev.bentixapp.com
db   -> MariaDB do ambiente dev
```

Notas:

- `web` e `api` usam a mesma imagem
- `web` recebe `NEXT_PUBLIC_APP_ENV=dev`
- `api` recebe `NEXT_PUBLIC_APP_ENV=local` para não se autoapontar para outra API pública

Como arrancar:

```
cd infra/environments/dev
cp .env.example .env
docker compose up -d --build
```

PROD

Topologia lógica desejada:

```
web -> bentixapp.com
api -> api.bentixapp.com
db -> MariaDB de produção
```

Topologia física atual no repositório:

```
app -> serve os papéis web + api
db -> MariaDB de produção
```

Isto significa:

- `bentixapp.com` e `api.bentixapp.com` apontam ambos para o mesmo serviço `app`
- a separação em produção é hoje lógica e por hostname, não por dois containers distintos

Como arrancar:

```
cd infra/environments/production
cp .env.example .env
docker compose up -d --build
```

Nginx

DEV

`infra/nginx/dev.conf.example` publica:

- `dev.bentixapp.com -> 127.0.0.1:3100`
- `api-dev.bentixapp.com -> 127.0.0.1:3101`

PROD

infra/nginx/production.conf.example publica:

- bentixapp.com -> 127.0.0.1:3300
- api.bentixapp.com -> 127.0.0.1:3300

Papel do Nginx

- termina TLS
- publica hostnames distintos
- encaminha tráfego para os serviços Docker corretos
- preserva headers como Host, X-Forwarded-For e X-Forwarded-Proto

Cloudflare

Registos DNS mínimos necessários:

Nome	Tipo sugerido	Destino	Uso
@	A ou CNAME	IP/hostname do VPS	frontend PROD
api	A ou CNAME	IP/hostname do VPS	API PROD
dev	A ou CNAME	IP/hostname do VPS	frontend DEV
api-dev	A ou CNAME	IP/hostname do VPS	API DEV

Recomendações práticas:

- usar proxy da Cloudflare ativo quando o ambiente já estiver público
- usar modo TLS Full (strict) quando existir certificado válido no origin
- garantir que os hostnames publicados no Nginx existem também no DNS

Fluxo resumido:



Fluxo de Deploy

Fluxo manual de referência:

```
Developer
↓
git commit
↓
git push
↓
git pull servidor
↓
docker compose build
↓
docker compose up
```

Equivalente atual em DEV

O repositório já inclui `/.github/workflows/deploy-dev.yml`, que automatiza o deploy de DEV:

```
Developer
↓
push na branch dev
↓
GitHub Actions
↓
npm ci
↓
npm run test:critical
↓
npm run build
↓
SSH para o servidor
↓
git fetch / reset
↓
docker compose up -d --build
```

Convenções do projeto

Como adicionar páginas

1. Criar a página em `app/<rota>/page.js`.
2. Se a página exigir sessão, usar `getSession()` e redirecionar quando necessário.

3. Para componentes client-side, chamar `frontend/controllers/*` em vez de fazer `fetch` ad hoc espalhado pela UI.

Como adicionar controllers

`server/controllers/*` deve:

- ler params, query e body
- obter sessão quando necessário
- transformar pedidos protegidos com `readProtectedRequestJson`
- chamar exatamente um ou poucos services
- devolver `jsonResponse(...)` ou equivalente
- não falar diretamente com Prisma

Como adicionar services

`server/services/*` deve:

- validar regras de negócio
- verificar permissões
- orquestrar chamadas a `lib/*`
- lançar `HttpError` quando a resposta HTTP precisa de um status explícito

Como adicionar endpoints REST

1. Criar `app/api/<rota>/route.js`.
2. Manter a rota fina, usando `toNextResponse` e `toNextErrorResponse`.
3. Criar ou reutilizar o controller em `server/controllers/*`.
4. Criar ou reutilizar o service em `server/services/*`.
5. Atualizar `server/docs/openapi-phase1.js` quando o endpoint for público/documentado.

Como adicionar chamadas frontend para a API

1. Criar função em `frontend/controllers/<dominio>-controller.js`.
2. Usar `apiFetch` ou `apiFetchJson`.
3. Deixar a resolução da URL sempre a cargo de `frontend/api/api-client.js`.

Como adicionar permissões

1. Declarar a permission key em `lib/permissions.js`.
2. Adicionar a key aos perfis adequados em `ACCESS_PROFILE_PERMISSION_KEYS`.
3. Aplicar a regra no service com `hasPermission(...)` ou `requireSessionPermissionService(...)`.
4. Se a nova rota/página tiver guard por path, atualizar o mapeamento em `lib/auth.js`.

Convenção de boundary HTTP

O padrão esperado para a API é:

```
route.js -> controller -> service -> lib -> lib/db -> Prisma
```

Qualquer desvio a este padrão deve ser excepcional e justificado.

Roadmap

Pontos ainda relevantes antes de uma operação de produção mais robusta:

- fechar runbooks e documentação operacional completa
- validar backup e restore ponta a ponta
- reforçar observabilidade, logs e alertas
- consolidar gestão de secrets fora de ficheiros locais
- automatizar rollout e rollback também para produção
- reduzir dependências legacy baseadas em ficheiros onde ainda existirem
- manter a separação futura do backend como evolução arquitetural, não como requisito imediato

Leitura prática do roadmap

A arquitetura atual já suporta operação em `LOCAL`, `DEV` e `PROD`, mas ainda há trabalho de maturidade operacional. O principal gap não está na existência de páginas ou endpoints, mas em:

- automação de operações
- monitorização
- disciplina de segredos
- rotinas de recuperação

Em resumo:

Arquitetura funcional: pronta

Maturidade operacional: ainda em evolução

Separação backend/frontend: preparada, mas ainda não executada